

IBM Quantum vs. Classical Distributed Systems: A Comparative Performance Analysis in High-Dimensional Data Processing

Jing Yi Choh

Faculty of Computing

Universiti Teknologi Malaysia (UTM)

Johor Bahru, Malaysia

Abstract—Due to the high dimensional data increase continue growth in stochastic financial modeling such as molecular simulation, classical distributed computing is facing physical and computational bottlenecks. This academic paper conducts a comparative performance analysis between conventional classical architectures such as the Apache Spark, and the quantum-based supercomputing system at IBM. This paper examines fundamental quantum physics such as superposition and tunable couplers, and the multilayer system architecture between the classical cloud space and the physical quantum hardware. Comparative analysis shows that IBM Quantum does not make classical systems obsolete but is a highly specialized computational accelerator. Although classical frameworks are still better at deterministic batch processing because they are more mature and have lineage-based fault tolerance, IBM Quantum can provide unparalleled parallelism to optimization tasks that are NP-hard. However, it is still limited in its wide usage due to its high sensitivity to errors, quantum decoherence and requires specialized cryogenic systems. In conclusion, this paper finds that the future of high-performance data processing is optimized hybrid orchestration, where quantum nodes are smoothly integrated into classical cloud pipelines in a fully optimized way to achieve the best overall computational performance.

Index Terms—IBM Quantum, high-dimensional data processing, distributed computing, performance benchmarking, hybrid orchestration

I. INTRODUCTION

In the contemporary landscape of high-performance data processing, the exponential growth of high-dimensional data has become a defining characteristic of industries such as global logistics, financial modelling, and molecular simulation (Wilson, A., & Anwar, M. R., 2024). To handle these huge workloads in computation, most enterprises have been historically using classical distributed systems and High-Performance Computing (HPC) architectures. These conventional paradigms handle information in series or parallel with classical bits, distributing tasks across computer nodes such as CPU and GPU to achieve a substantial increase in throughput (Thomadakis & Chrisochoides, 2024).

Nevertheless, data dimensionality and complexity of algorithms continue to increase, classical distributed architectures are encountering fundamental bottlenecks. For complex operations of linear algebra, and NP-hard optimization problems, classical systems have severe memory bandwidth

constraints, large energy consumption, and exponentially increase in processing latency (Alonso et al., 2025). This means that conventional frameworks are approaching their physical and algorithmic limits when processing massive quantities of multidimensional problem spaces efficiency.

The solution of processing limitations is essential for organizations that require real time optimization and complex system modelling. Quantum computing is a paradigm shift in computational architecture. Using quantum mechanical concepts like superposition and entanglement, quantum processors have the theoretical ability to measure exponential number of data states simultaneously (Rietsche et al., 2022). Understanding of how this technology can be put into practical use is essential to data engineers who want to overcome the limitations of classical computing and achieve exponential performance improvements.

The main objective of this paper is to critically compare and contrast the processing performance of IBM Quantum and classical distributed systems, specifically in the case of high dimensional data workloads. By examining processing models, execution speeds, and system constraints, this paper will analyse the current quantum advantage threshold.

II. CONTENT DISCUSSION

A. Core Mechanisms and Design Principle

IBM Quantum is an advanced cloud-based technology that grants organizations and researchers access to real quantum computers and simulators (M. Steffen et al., 2011). In its simplest form, the technology works with quantum bits, or qubits, that instead of being restricted to 0 or 1 like classical bits, can have both 0 and 1 at the same time as a process called quantum superposition (Zohuri, B., & Rahmani, F. M., 2020). With superposition and quantum entanglement, software applications run carefully coordinated processes of quantum logic gates to act on exponentially large sets of information, enabling a large increase in the amount of computational power with an increase in the number of qubits.

IBM's architecture is built based on Quantum-Centric Supercomputing. IBM quantum computers include 127-qubit Eagle, 156-qubit Heron r2 and 120-qubits Nighthawk processors (Döbler, P., & Jattana, M. S., 2025). These processors are designed to prioritize quality and connectivity between

qubits. It used a critical feature called tunable couplers. This hardware feature allows the system to shut down interactions between qubits when they are idle which drastically reduces interference from neighbouring qubits often referred to as crosstalk. Additionally, the qubits are arranged in a heavy hexagonal shape. This structural design limits the number of connections between each qubit which helps reduce errors while still allowing for complex entanglement.

IBM's architecture is guided by two core design principles, modularity and Quantum-Classical Integration. First, IBM designs its systems to be modular instead of trying to build one giant and monolithic chip. Thus, it allows multiple quantum processors to be linked together and makes it much easier to scale up computational power over time (Ferrari & Amoretti, 2023). Second, IBM shapes these systems in such a way that they can be integrated with conventional systems. Instead of vertically replacing the classical servers, the quantum processor is a specialized accelerator. The routine data routing and heavy lifting are carried out by the classical CPUs and GPUs, the quantum processor is only invoked when it is needed to perform the complex mathematical problems, which cannot be solved by conventional machines (Rached et al., 2024).

B. System Architecture

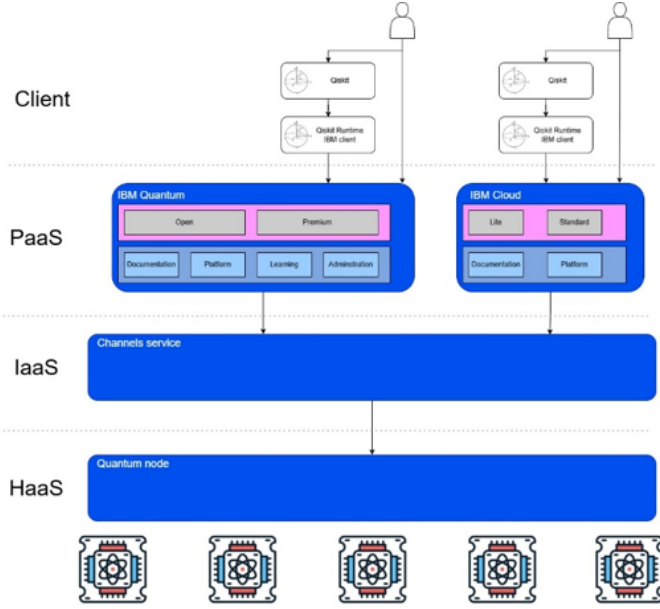


Fig. 1. High-level view of IBM Quantum software architecture (adapted from Khan ET AL., 2023)

The IBM quantum employs a highly organized multi-layered software system that is designed to smoothly interconnect between classical software environments and physical quantum hardware. As shown in Fig. 1, the ecosystem is divided into four major operation levels to ensure scalable and reliable quantum utility.

The Client layer is at the top, which allows the researchers and developers to interact with the system through the Qiskit

framework and the Qiskit Runtime IBM client. This user facing environment grabs quantum circuits up to the Platform as a Service (PaaS) layer. The PaaS layer operates in both the IBM Quantum and standard IBM Cloud environments, taking care of administrative operations, learning materials, and platform documentation.

The workloads are then passed to the Infrastructure as a Service (IaaS) layer. This layer is a dedicated channels service, which is effective in queuing, routing, and controlling the flow of data between the classical cloud and the quantum systems. Lastly, the instructions are sent to the Hardware as a Service (HaaS) layer. The physical quantum nodes are stored in this bottom level, which is the dilution refrigerators and superconducting processors in which the quantum circuits are physically implemented.

C. Performance Characteristics

IBM uses three primary metrics to measure performance: Scale, Quality, and Speed.

TABLE I
PERFORMANCE CHARACTERISTICS OF IBM QUANTUM (ADAPTED FROM ABUGHANEM, 2025)

Metric	Definition	2026 Capability
Scale	The number of qubits which indicates the amount of information can encode in the quantum system	1,092 qubits
Quality	Measured by Quantum Volume which indicates quality of circuits and how faithfully circuits are implemented in hardware	7,500 gates
Speed	Measured by CLOPS (Circuit Layer Operations Per Second) which indicates how many circuits can run on hardware in a given time	Up to 250,000 CLOPS on System Two.

D. Real World Case Study

In the global financial services sector, algorithmic trading requires the continuous processing of high-dimensional data, including real-time market conditions and complex risk estimates. To address this, HSBC collaborated with IBM to apply a combination of quantum computing and classical computational resources to analyse HSBC's real-world, production-scale bond trading data. The hybrid quantum-classical methods to process information employed by the HSBC-IBM team yielded up to a 34% improvement over purely classical techniques for predicting which trades would be completed. This proves that contemporary quantum computers can solve real-world business problems at scale. It demonstrates the emerging potential of quantum computing as a complementary exploratory tool in quantitative finance and encourages further applied industry research towards practical trading applications (Ciceri. et al, 2025).

E. Comparative Performance of Apache Spark vs. IBM Quantum

Before quantum supercomputing emerged, classical distributed systems like Apache Spark were used to process large

volumes of data. Apache Spark is a highly mature and unified analytics engine for large scale distributed data processing. It uses Resilient Distributed Datasets (RDDs) as a way of storing data directly in random access memory (RAM) in a cluster of classical nodes (Shaikh et al., 2019). The strategy removes the extreme disk I/O latency of the older classical systems, enabling Spark to deliver high speed throughput and lineage-based fault tolerance.

Although classical system has immense operational maturity, it encounters severe physical and computational bottlenecks when processing high complexity of dimensional data. Classical systems rely on bits (0 or 1) and must process high throughput with step-by-step operations. Thus, it face physical and computational bottlenecks, especially high energy usage and processing time when trying to model high-dimensional complex data. The key limitations are overcome by IBM Quantum based on a radical change of architecture. IBM processors can inherently explore and map large computational spaces at once by using qubits that exist in a state of quantum superposition.

TABLE II
COMPARATIVE PERFORMANCE OF APACHE SPARK VS. IBM QUANTUM

Criteria	Conventional System (Apache Spark)	Modern System (IBM Quantum)
Basic Unit	Bit (0 or 1)	Qubit (can be 0 and 1 simultaneously)
Processing Style	Step-by-step or limited parallelism	Huge parallelism through superposition
Energy	High power consumption	Highly specialized infrastructure (dilution refrigerators)
Error Sensitivity	Low error rates, easy to correct	High error rates, needs advanced correction
Fault Tolerance	RDD lineage graphs for exact data recovery	Susceptible to quantum noise and decoherence.
Best Use Case	Batch and real-time streaming workloads	NP-hard optimization, stochastic financial modeling, molecular simulation

However, IBM Quantum will not replace the classical systems but act as a specialized accelerator. Apache Spark has been able to sustain a strong operational advantage in system maturity and reliability. It is the better and less expensive option due to its low error sensitivity and ability to recover data precisely, which is possible through Resilient Distributed Dataset (RDD) lineage graphs. In contrast, contemporary IBM Quantum systems are very sensitive to environmental noise and quantum decoherence, require sophisticated error correction algorithms and very specialized infrastructure including dilution refrigerators. Thus, the actual performance benefit of IBM Quantum is limited to limited applications such as solving NP-hard optimization problems, running stochastic financial modeling, and modeling the structure of a molecule where classical step-by-step based processing is computationally infeasible.

III. CONCLUSION

This paper has discussed the architectural shift from classical distributed architecture to quantum-based supercomputing at IBM to address the increasing bottlenecks in high dimensional data processing. IBM Quantum uses quantum superposition and entanglement, combined with cutting-edge hardware such as tunable couplers and heavy-hexagonal qubit arrays to overcome the sequential limitations of conventional CPU and GPU clusters to provide unprecedented computational parallelism.

A critical insight made derived from the comparative analysis is that IBM Quantum cannot make classical systems such as Apache Spark redundant. Instead, it functions as a specialized computational accelerator. Although classical architectures are still the most efficient and dependable solution to deterministic batch processing and standard data streaming, quantum systems are uniquely effective at solving exponentially complex problems, including NP-hard optimization problems, molecular simulations, and stochastic financial modelling problems where the conventional systems fail.

Although the existing version of quantum technology has a potential of paradigm shift, it is limited by major physical weaknesses. IBM quantum processors are highly vulnerable to noise and quantum decoherence of the environment. They currently lack mature, with fault tolerance based on lineage as in classical distributed data systems and need very specialized cryogenic infrastructure to operate. Therefore, the future research should highly prioritise the development of effective quantum error-correction algorithms and robust hardware architectures. Moreover, as the industry advances to practical quantum utility, new advances should be focus on optimizing hybrid orchestration framework. To achieve overall scalability in the future generation of data science, it will be crucial to investigate how to seamlessly incorporate quantum nodes in the existing classical cloud pipelines.

REFERENCES

- [1] AbuGhanem, M. (2025). IBM quantum computers: evolution, performance, and future directions. *The Journal of Supercomputing*, 81(5). <https://doi.org/10.1007/s11227-025-07047-7>
- [2] Alonso, M., Rodriguez, G. R., Diez-Valle, P., Garbayo, A., Garcia-Santiago, X., & Gil, G. B. (2025). Modeling energy communities: A case study of quantum approximate optimization on a superconducting processor. *IEEE Access*, 13, 106140-106154. <https://doi.org/10.1109/ACCESS.2025.3578780>
- [3] Ciceri, A., Cottrell, A., Freeland, J., Fry, D., Hirai, H., Intallura, P., Kang, H., Lee, C., Mitra, A., Ohno, K., Pemmaraju, D., Proissl, M., Quanz, B., Rajan, D., Shimada, N., & Yograji, K. (2025, September 22). Enhanced fill probability estimates in institutional algorithmic bond trading using statistical learning algorithms with quantum computers. *arXiv.org*. <https://arxiv.org/abs/2509.17715>
- [4] Döbler, P., & Jattana, M. S. (2025, July 4). A Survey on Integrating Quantum Computers into High Performance Computing Systems. *arXiv.org*. <https://arxiv.org/abs/2507.03540>
- [5] Ferrari, D., Carretta, S., & Amoretti, M. (2023). A modular quantum compilation framework for distributed quantum computing. *IEEE Transactions on Quantum Engineering*, 4, 1-13. <https://doi.org/10.1109/tqe.2023.3303935>
- [6] Khan, A. A., Ahmad, A., Waseem, M., Liang, P., Fahmideh, M., Mikkonen, T., & Abrahamsson, P. (2023). Software architecture for quantum computing systems: A systematic review. *Journal of Systems and Software*, 201, 111682. <https://doi.org/10.1016/j.jss.2023.111682>

- [7] M. Steffen, D. P. DiVincenzo, J. M. Chow, T. N. Theis and M. B. Ketchen, "Quantum computing: An IBM perspective," in *IBM Journal of Research and Development*, vol. 55, no. 5, pp. 13:1-13:11, Sept.-Oct. 2011, doi: 10.1147/JRD.2011.2165678.
- [8] Rached, S. B., Agudo, L. L., Rodrigo, S., Bandic, M., Feld, S., Hans, V. S., Alarcón, E., Almudéver, C. G., & Abadal, S. (2023, October 3). Characterizing the Inter-Core Qubit traffic in Large-Scale Quantum Modular architectures. *arXiv.org*. <https://arxiv.org/abs/2310.01921>
- [9] Rietsche, R., Dremel, C., Bosch, S., Steinacker, L., Meckel, M., & Leimeister, J. (2022). Quantum computing. *Electronic Markets*, 32(4), 2525-2536. <https://doi.org/10.1007/s12525-022-00570-y>
- [10] Shaikh, I. Mohiuddin, Y. Alufaisan and I. Nahvi, "Apache Spark: A Big Data Processing Engine," 2019 2nd IEEE Middle East and North Africa COMMUNICATIONS Conference (MENACOMM), Manama, Bahrain, 2019, pp. 1-6, doi: 10.1109/MENACOMM46666.2019.8988541.
- [11] Thomadakis, P., & Chrisochoides, N. (2024). Runtime support for CPU-GPU high-performance computing on distributed memory platforms. *Frontiers in High Performance Computing*, 2. Article 1417040. <https://doi.org/10.3389/fhpcp.2024.1417040>
- [12] Wilson, A., & Anwar, M. R. (2024). The future of adaptive machine learning algorithms in high-dimensional data processing. *International Transactions on Artificial Intelligence*, 3(1), 97-107.
- [13] Zohuri, B., & Rahmani, F. M. (2020). What is quantum computing and how it works. *Journal of Material Sciences & Manufacturing Research*, 1-5. [https://doi.org/10.47363/jmsmr/2020\(2\)104](https://doi.org/10.47363/jmsmr/2020(2)104)